

INSTITUTO INFNET



ENGENHARIA DE DADOS E IA

PROJETO INTEGRADOR DE FRONT-END COM FRAMEWORKS

Teste de Performance 5

Alunos: Marcus Vinícius dos Santos Liberato, Arthur Bezerra de Souza e Louriane Bueno.

Projeto: Plataforma Web de Compra e Venda de Criptomoedas

Equipe: Marcus (Front-end), Louriane (UI/UX), Arthur (Back-end)

25/03/2026

1. DEFINIÇÃO DO PROJETO

O projeto escolhido para desenvolvimento é uma **Plataforma Web de Compra e Venda de Criptomoedas**. A aplicação permitirá aos usuários realizar operações de compra e venda de diferentes criptomoedas, acompanhar cotações, visualizar o histórico de transações e gerenciar seu portfólio digital.

A escolha desse projeto se justifica pela relevância atual do mercado de criptomoedas e pela oportunidade de trabalhar com consumo de dados em tempo real, integração com APIs externas e construção de interfaces responsivas e interativas. Além disso, o projeto favorece a aplicação de conceitos fundamentais do desenvolvimento front-end com frameworks modernos, especialmente com o uso de React.

A plataforma será desenvolvida com foco na experiência do usuário, oferecendo uma interface intuitiva tanto para iniciantes quanto para usuários com maior familiaridade com operações de mercado. Para isso, serão utilizados recursos visuais como cards informativos, indicadores dinâmicos, gráficos e dashboards organizados de forma clara e responsiva.

2. DEFINIÇÃO DE ESCOPO E OBJETIVOS

Objetivo principal

Desenvolver uma aplicação web responsiva e intuitiva que permita aos usuários realizar operações de compra e venda de criptomoedas, com acesso a dados de mercado e estrutura preparada para evolução funcional.

Funcionalidades Específicas:

- **Autenticação e segurança:** sistema de login e cadastro, persistência de sessão e proteção de rotas.
- **Dashboard Principal:** visualização das principais criptomoedas, com indicadores de preço, variação e comportamento do mercado.
- **Sistema de compra e venda:** interface para simulação e confirmação de operações.
- **Carteira digital:** gerenciamento do portfólio do usuário, com visão consolidada dos ativos.
- **Consumo de API externa:** integração com serviço de mercado para obtenção de cotações atualizadas.
- **Responsividade total:** adaptação da interface para desktop, tablet e smartphone.
- **Interatividade:** atualização dinâmica dos componentes conforme mudança de estado.
- **Notificações e feedback visual:** mensagens de sucesso, erro e carregamento.
- **Tema visual:** possibilidade de uso de modo escuro e claro.

3. GESTÃO ÁGIL DO PROJETO (SCRUM)

Metodologia Adotada

O projeto será desenvolvido com base no framework Scrum, priorizando entregas incrementais, acompanhamento contínuo do progresso e adaptação com base em feedbacks. O trabalho será organizado em sprints de duas semanas, contemplando planejamento, execução, revisão e retrospectiva.

Definition of Ready (DoR)

Um item do Product Backlog será considerado pronto para entrar em uma sprint quando atender aos seguintes critérios objetivos:

- História de usuário escrita no formato “Como..., eu quero..., para...”
- Critérios de aceitação claros e testáveis
- Prioridade definida pelo Product Owner
- Estimativa de esforço atribuída (Story Points)
- Dependências identificadas
- Dados, APIs e recursos necessários disponíveis
- Interface ou fluxo previamente validado no Figma (quando aplicável)

Somente itens que atendam a todos esses critérios poderão ser selecionados durante a Sprint Planning.

Definition of Done (DoD)

Uma entrega será considerada concluída somente quando atender aos seguintes critérios:

- Funcionalidade implementada conforme critérios de aceitação
- Código revisado e integrado à branch principal
- Testes básicos executados
- Interface validada visualmente
- Ausência de erros críticos de execução
- Integração funcional com o restante do sistema
- Documentação mínima atualizada
- Incremento disponível para demonstração na Sprint Review

Política de Refinamento do Backlog

O Product Backlog Refinement será realizado semanalmente, com participação de toda a equipe. Nessa atividade serão executadas:

- revisão de prioridades;
- detalhamento de histórias futuras;
- decomposição de histórias grandes em tarefas menores;
- estimativas de esforço;
- identificação de riscos e dependências técnicas.

4. CRONOGRAMA DO PROJETO

Calendário Geral do Projeto

Período do projeto: 20/01/2026 a 16/03/2026.

Duração: 8 semanas.

Sprints: 4 sprints de 2 semanas.

Sprint 1 – Planejamento e Fundação

20/01 a 02/02/2026

Tarefa	Responsável	Esforço (h)	Dependência	Critério de Aceite
Criar repositório e CI/CD	Marcus	6h	—	Repositório estruturado e disponível
Estrutura inicial React/Next	Marcus	10h	Repo criado	Aplicação renderizando página base
Wireframes no Figma	Louriane	12h	—	Fluxos aprovados
Definir arquitetura back-end	Arthur	10h	—	Documento técnico validado

Marco M1: protótipo navegável + ambiente configurado.

Critério de Aceite: aplicação inicial funcionando e fluxos validados.

Sprint 2 – MVP Visual

03/02 a 16/02/2026

Tarefa	Responsável	Esforço (h)	Dependência	Critério de Aceite
Página Login	Marcus	8h	UI aprovada	Login renderizado
Dashboard com Recharts	Marcus	12h	API disponível	Gráfico exibindo informações
API de autenticação	Arthur	10h	Arquitetura definida	Fluxo básico validado
Ajustes responsivos	Louriane	8h	Páginas criadas	Interface validada em mobile

Marco M2: MVP navegável.

Critério de Aceite: fluxo básico entre Login e Dashboard funcional.

Sprint 3 – Integração

17/02 a 02/03/2026

- Integração do front-end com serviço de mercado
- Estrutura do fluxo de compra e venda
- Testes do fluxo principal

Marco M3: Sistema integrado

Critério de Aceite: usuário acessa o dashboard, visualiza dados atualizados e interage com a interface de operação.

Sprint 4 – Finalização e Deploy

03/03 a 16/03/2026

- Otimizações de performance
- Preparação para publicação
- Documentação final

Marco M4: Aplicação publicada

Critério de Aceite: Sistema online, documentado e demonstrável

Estratégias para Cumprimento de Prazos:

Estratégia	Descrição
Daily Scrum (15 min/dia)	Reunião breve para identificação de bloqueios e alinhamento
Kanban Board (Trello/Jira)	Organização das tarefas em “A Fazer”, “Em Progresso” e “Concluído”
Sprint Review	Demonstração das entregas e coleta de feedback
Sprint Retrospective	Identificação de pontos de melhoria
Versionamento (GitHub)	Controle de versões com GitHub
Documentação Contínua	Registro das atividades ao longo das sprints
Testes Automatizados	Redução de erros e retrabalho

5. CRIAÇÃO DE HISTÓRIAS DE USUÁRIO

Artefatos do Scrum

Product Backlog

Lista priorizada das funcionalidades, melhorias e correções necessárias para o produto. Será mantido e atualizado pelo Product Owner, incluindo itens como autenticação, dashboard, integração com API, compra e venda, carteira digital e testes.

Sprint Backlog

Subconjunto do Product Backlog selecionado para cada sprint, decomposto em tarefas menores e distribuído entre os integrantes da equipe.

Incremento

Produto potencialmente entregável ao final de cada sprint. Ao final da Sprint 2, por exemplo, a equipe prevê uma interface funcional com páginas principais já estruturadas.

Papéis do Scrum

Product Owner (Arthur)

Responsável por priorizar o backlog e orientar a visão do produto.

Scrum Master (Louriane)

Responsável por facilitar o processo, acompanhar impedimentos e apoiar a organização da equipe.

Time de Desenvolvimento (Marcus, Louriane e Arthur)

Equipe multifuncional responsável pela construção incremental da solução.

Eventos do Scrum

Sprint Planning

Reunião de definição dos objetivos da sprint e dos itens que serão desenvolvidos.

Daily Scrum

Reunião diária breve para alinhamento do andamento do trabalho.

Sprint Review

Demonstração do incremento produzido e coleta de feedback.

Sprint Retrospective

Reflexão sobre o processo e definição de ações de melhoria.

5.1 Tabela comparativa das histórias de usuário

<u>Elemento</u>	<u>História original</u>	<u>História atualizada</u>	<u>Melhoria aplicada</u>
Perfil do usuário	Usuário genérico	Como investidor da plataforma, eu quero visualizar cotações de criptomoedas em tempo real, para acompanhar o mercado e apoiar minhas decisões de investimento.	Definição clara do ator da história
Objetivo da história	Visualizar informações sobre criptomoedas	Como investidor da plataforma, eu quero visualizar cotações e variações dos ativos, para analisar o comportamento do mercado antes de comprar ou vender.	Objetivo funcional mais preciso
Contexto de uso	Não especificado	Como usuário autenticado da Dashboard, eu quero acompanhar os dados de mercado em uma interface responsiva, para consultar informações atualizadas em diferentes dispositivos.	Inclusão do contexto de uso
Benefício esperado	Acompanhar investimentos	Como investidor da plataforma, eu quero consultar minha carteira e os dados de mercado, para tomar decisões mais informadas sobre meus ativos.	Valor entregue explicitado
CrITÉRIOS de aceitação	Implícitos	Como usuário da plataforma, eu quero visualizar dados atualizados, mensagens de erro e estados de carregamento, para utilizar o sistema com clareza e previsibilidade.	Maior facilidade de validação
Requisitos técnicos	Não especificados	Como equipe de desenvolvimento, eu quero integrar a interface a uma API externa com tratamento de erros, para garantir atualização confiável dos dados exibidos.	Melhor alinhamento entre requisito e implementação
Aderência ao Scrum	História ampla	Como investidor da plataforma, eu quero visualizar cotações em tempo real, para acompanhar o mercado de forma prática e apoiar minhas decisões.	Maior conformidade com o padrão Scrum

6. PLANEJAMENTO DO DESENVOLVIMENTO FRONT-END

a) Definição da stack tecnológica

A implementação da versão web do projeto foi planejada com base em uma stack compatível com o escopo acadêmico e com a arquitetura efetivamente utilizada no protótipo:

- **Front-end web:** React com Vite
- **Gerenciamento de estado:** hooks nativos do React, com possibilidade de evolução para Context API ou custom hooks mais especializados
- **Estilização:** CSS próprio com abordagem mobile-first
- **Gráficos:** Recharts
- **Integração HTTP:** camada de serviços para consumo de API externa

Como perspectiva futura de evolução, o projeto também considera a possibilidade de adaptação para ambiente mobile com React Native, aproveitando a lógica de negócio e a estrutura de componentização.

b) Configuração do ambiente

- Criação do repositório no GitHub
- Estruturação do projeto React com Vite
- Organização da base de arquivos
- Configuração do ambiente local
- Preparação das dependências necessárias para execução do projeto

c) Arquitetura do projeto

- Definição de estrutura de pastas, componentes e páginas
- Criação de componentes reutilizáveis
- Organização de estilos compartilhados
- Separação entre lógica de dados, navegação e apresentação
- Preparação da aplicação para integração progressiva com serviços externos

MARCOS PRINCIPAIS (MILESTONES):

Marco Entregável Semana

MARCO	ENTREGÁVEL	SEMANA
M1 - Fundação	Protótipo e estrutura inicial aprovados	Semana 2
M2 - MVP Visual	Interface funcional com navegação básica	Semana 4
M3 - Integração	Front-end conectado a dados externos	Semana 6
M4 - Release	Aplicação pronta para apresentação	Semana 8

Critérios de qualidade

- Interface responsiva e estável
- Organização coerente dos componentes
- Ausência de erros críticos de execução
- Estrutura preparada para evolução funcional
- Clareza visual e consistência de navegação

7. INTERATIVIDADE NA APLICAÇÃO WEB E PERSPECTIVA MOBILE

No estágio atual do projeto, a interatividade implementada refere-se à **versão web** da aplicação. A interface foi construída em React para oferecer atualização dinâmica de componentes, feedback visual ao usuário e reatividade a mudanças de estado e ao retorno de dados externos.

Na versão web, foram previstos e descritos os seguintes comportamentos interativos:

- autenticação com validação de formulário;
- mensagens visuais de sucesso e erro;
- atualização automática dos dados exibidos no dashboard;
- renderização reativa de cards e indicadores de mercado;
- resposta visual a estados de carregamento e falha;
- atualização de interface a partir de ações do usuário e retorno da API.

Entretanto, a **transformação da aplicação em app mobile nativo ainda não foi implementada nesta entrega**. Embora o projeto tenha sido concebido com preocupação de responsividade para telas menores, isso não equivale à entrega de uma aplicação nativa mobile.

Da mesma forma, os recursos de interação específicos de ambiente móvel, como **pull-to-refresh, swipe actions, gestos touch e feedback tátil**, ainda **não fazem parte da versão efetivamente implementada**. Esses elementos permaneceram como diretrizes de evolução do produto e deverão ser tratados em etapa posterior, com arquitetura, ambiente e código próprios para mobile.

Assim, para fins de avaliação desta entrega, a interatividade comprovadamente implementada corresponde à **interface web responsiva e reativa**, não a uma versão mobile nativa.

8. FRAMEWORKS: REACTJS E REACT NATIVE

Os frameworks ReactJS e React Native foram considerados como referência tecnológica do projeto por possibilitarem componentização, reuso de lógica e construção de interfaces modernas.

ReactJS para aplicação web

No estágio atual do projeto, o ReactJS é o framework efetivamente utilizado na construção da aplicação web. Seu uso permite:

- criação de componentes reutilizáveis;
- atualização reativa da interface;

- separação entre lógica e apresentação;
- melhor organização da aplicação;
- integração progressiva com dados externos.

React Native como evolução da solução

O React Native é tratado como uma possibilidade de expansão futura da plataforma para dispositivos móveis. Sua adoção seria vantajosa por permitir:

- reaproveitamento de parte da lógica de negócio;
- adaptação do fluxo para interações touch;
- desenvolvimento multiplataforma;
- manutenção unificada de uma base tecnológica alinhada ao ecossistema React.

Benefícios combinados

Benefício	ReactJS (Web)	React Native (Mobile)
Reuso conceitual	Componentização e hooks	Mesma lógica de organização
Ecossistema	Grande disponibilidade de bibliotecas	Suporte a recursos nativos
Manutenção	Estrutura modular	Evolução multiplataforma
Interatividade	Atualização reativa da interface	Gestos e interações nativas

A escolha do ReactJS para a versão entregue e do React Native como possibilidade de evolução mantém o projeto tecnicamente coerente e aderente à proposta de desenvolvimento com frameworks.

9. INSTRUÇÕES PARA EXECUÇÃO DO PROJETO

Passo a passo para abrir e executar o projeto no VS Code:

1) Extrair o arquivo .zip

- Fazer download do arquivo do projeto;
- extrair o conteúdo para uma pasta local;
- confirmar a presença dos arquivos principais, como public, src, index.html, package.json, pnpm-lock.yaml, vite.config.js e README.md.

2) Abrir a pasta correta no VS Code

- Abrir o VS Code;
- selecionar **Arquivo > Abrir Pasta**;
- escolher a pasta raiz do projeto, isto é, a pasta que contém o arquivo package.json.

3) Abrir o terminal

- No VS Code, acessar **Terminal > Novo Terminal**;
- verificar se o terminal está posicionado na pasta do projeto.

4) Instalar as dependências

No terminal do VS Code execute os seguintes comandos **NA PASTA DO PROJETO** para instalar o pnpm:

- a. `npm install -g pnpm`
- b. `pnpm install`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> npm install -g pnpm

added 1 package in 8s

1 package is looking for funding
  run `npm fund` for details
● PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> npm install -g pnpm

changed 1 package in 5s

1 package is looking for funding
  run `npm fund` for details
○ PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> █
```

- c. Ainda DENTRO DA PASTA DO PROJETO, rode: **pnpm install**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> pnpm install
+ eslint-plugin-react-refresh 0.4.26
+ globals 15.15.0 (17.1.0 is available)
+ vite 6.4.1 (7.3.1 is available)

Warning
-----
Ignored build scripts: esbuild@0.25.12.
Run "pnpm approve-builds" to pick which dependencies should be allowed to run scripts.

Done in 40.9s using pnpm v10.28.1
○ PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> █
```

- d. Verifique se o Vite está no projeto com o seguinte comando: **pnpm list vite**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> pnpm list vite
Legend: production dependency, optional only, dev only

react-vite@0.0.0 C:\Users\marcu\Downloads\Plataforma_Criptmoeda (PRIVATE)

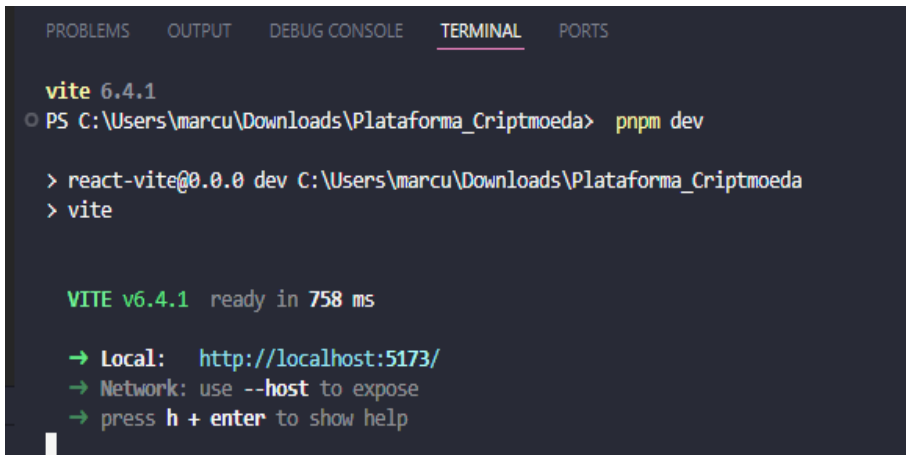
devDependencies:
vite 6.4.1
○ PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> █
```

Se aparecer algo como “**vite@x.x.x**”, como na imagem acima, está tudo ok.

5) Rodar o projeto

No terminal do VS Code execute o seguinte comando para rodar o projeto:

- a. **pnpm dev**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

vite 6.4.1
PS C:\Users\marcu\Downloads\Plataforma_Criptmoeda> pnpm dev

> react-vite@0.0.0 dev C:\Users\marcu\Downloads\Plataforma_Criptmoeda
> vite

VITE v6.4.1 ready in 758 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

b.

6) Abrir no navegador

No terminal vai aparecer algo parecido com:

- Local: <http://localhost:5173/>

Abra esse endereço no navegador.

10. ESTRATÉGIA DE REUTILIZAÇÃO DE COMPONENTES

A reutilização de componentes foi adotada como estratégia central para garantir organização, padronização visual e facilidade de manutenção. A interface foi planejada com base em componentes reaproveitáveis, nos quais cada bloco visual segue uma estrutura consistente e pode ser renderizado dinamicamente a partir de dados.

A aplicação utiliza React com foco em componentes funcionais e em renderização orientada por dados. Em vez de criar manualmente cada card ou seção repetida, foram definidos conjuntos de dados estruturados em arrays de objetos, percorridos com `map()`, permitindo a geração automática de elementos visuais de forma consistente e escalável.

Outro ponto importante é o reaproveitamento de padrões visuais por meio de classes CSS compartilhadas. Elementos como botões, cards e grids seguem estilos comuns que garantem identidade visual e reduzem duplicação de código.

Além disso, a lógica de controle da aplicação foi organizada de modo a separar o gerenciamento de estado da apresentação visual, favorecendo a evolução do projeto para uma arquitetura mais modular. Essa estratégia torna a aplicação mais flexível, consistente e preparada para novas funcionalidades.

11. ARTEFATO: BACKLOG (PRÓXIMAS ENTREGAS – PLATAFORMA DE CRIPTOMOEDA)

ID	Tipo	Atividades	Objetivo	Prioridade	Crítérios de aceite	Dependências
BL-01	Funcionalidade	Autenticação com persistência de sessão	Permitir acesso seguro	Alta	Login válido, logout funcional e proteção de rota	BL-02
BL-02	Segurança	Estrutura básica de segurança	Evitar acesso indevido	Alta	Rotas privadas e tratamento de sessão inválida	—
BL-03	Funcionalidade	Tela de carteira	Exibir ativos e saldo total	Alta	Lista responsiva e cálculos consistentes	BL-06
BL-04	Funcionalidade	Histórico de transações	Permitir auditoria do usuário	Alta	Filtros, ordenação e navegação consistente	BL-06
BL-05	Funcionalidade	Fluxo de compra e venda	Permitir operar ativos	Alta	Validação, confirmação e atualização visual	BL-03, BL-04
BL-06	Integração	Camada de API e tratamento de erros	Isolar dados da interface	Alta	Services separados, loading e error padronizados	—
BL-07	Refactoring	Normalização de componentes e pastas	Melhorar manutenção	Alta	Estrutura coerente e componentes reutilizáveis	—
BL-08	Refactoring	Custom hooks	Reuso e redução de duplicação	Média	Hooks com loading, error e data	BL-06
BL-09	Qualidade	Validações de formulários	Reduzir erros de uso	Média	Mensagens claras e bloqueio de envio inválido	BL-01, BL-05
BL-10	UX/UI	Loading, empty state e skeletons	Melhorar experiência	Média	Ausência de tela vazia e feedback visual	BL-06
BL-11	UI Responsiva	Ajustes para telas diversas	Melhorar responsividade	Média	Sem overflow e com adaptação de layout	—
BL-12	Observabilidade	Logs e monitoramento	Facilitar depuração	Média	Captura padronizada de erros	BL-06
BL-13	Testes	Testes unitários	Evitar regressões	Alta	Execução consistente e cobertura inicial definida	BL-08
BL-14	Testes	Testes de componentes	Garantir estabilidade da UI	Alta	Renderização e interações principais validadas	BL-07

ID	Tipo	Atividades	Objetivo	Prioridade	Crítérios de aceite	Dependências
BL-15	Testes	Testes E2E	Validar jornada principal	Média	Fluxos críticos automatizados	BL-01, BL-05
BL-16	Performance	Otimizações	Melhorar carregamento	Média	Redução de re-renderizações e melhor fluidez	BL-07
BL-17	Segurança	Hardening de inputs	Reduzir risco em campos	Média	Tratamento seguro da entrada de dados	BL-09
BL-18	Documentação	README técnico e guia do projeto	Facilitar manutenção	Média	Execução e arquitetura documentadas	—
BL-19	Funcionalidade	Página de ativo	Exibir detalhe por criptomoeda	Média	Dados e indicadores por ativo	BL-06
BL-20	Qualidade	Padronização de estilo	Consistência de código	Média	Lint e formatação padronizados	BL-07

12. ENUMERAÇÃO DAS MUDANÇAS DE REQUISITOS PROPOSTAS PELO PROFESSOR E REVISÃO DO PLANEJAMENTO ANTERIOR

Com base no retorno recebido sobre a entrega anterior, foram identificadas e consolidadas as seguintes mudanças de requisitos propostas pelo professor para o desenvolvimento deste TP5:

1. explicitar quais aspectos do trabalho anterior foram avaliados e quais limitações foram identificadas, em vez de apenas mencionar genericamente que houve feedback;
2. relacionar de forma objetiva cada feedback recebido à respectiva modificação realizada, garantindo rastreabilidade entre a observação do professor e o ajuste incorporado ao projeto e ao relatório;
3. indicar claramente qual componente foi extraído da antiga estrutura de página única, evitando uma descrição abstrata da componentização;
4. justificar tecnicamente a decisão arquitetural de extração desse componente, demonstrando porque a mudança melhora a organização da aplicação;
5. demonstrar a reutilização concreta do componente extraído entre páginas distintas, e não apenas dentro de uma única tela;
6. comprovar textualmente a implementação efetiva da integração com API, deixando claro que o consumo de dados reais não permaneceu apenas como meta futura;
7. formalizar o fluxo reativo de dados entre serviço, estado e componentes, explicando como a interface reage às respostas da API e às ações do usuário;
8. revisar o backlog com priorização objetiva, considerando valor de negócio, risco técnico e aderência à expectativa do cliente;
9. distinguir com maior clareza o que foi efetivamente desenvolvido no plano e o que permanecerá como entrega do plano atual.

A partir dessa enumeração, o presente plano foi reorganizado para responder diretamente a cada uma dessas exigências. Assim, o relatório deixa de apresentar apenas melhorias gerais e passa a demonstrar, de forma verificável, quais mudanças foram solicitadas e como cada uma delas foi incorporada à documentação, à arquitetura e ao planejamento do projeto.

Em resposta ao primeiro e ao segundo pontos, foi criada esta seção específica de revisão, na qual os feedbacks recebidos são apresentados de forma explícita e relacionados às modificações realizadas. Em resposta ao terceiro, ao quarto e ao quinto pontos, o relatório passa a identificar o componente extraído da antiga estrutura centralizada, justificar tecnicamente essa decisão e demonstrar sua reutilização entre páginas distintas. Em resposta ao sexto ponto, o texto passa a detalhar a integração efetiva com API externa, descrevendo a existência de uma camada de serviços, o consumo de dados de mercado, o tratamento de carregamento, o tratamento de falhas e a atualização automática da interface com base nas respostas recebidas. Em resposta ao sétimo ponto, o relatório formaliza o fluxo reativo da aplicação, mostrando que a interface responde a alterações de estado decorrentes tanto da interação do usuário quanto do retorno de dados externos. Por fim, em resposta ao oitavo e ao nono pontos, o backlog foi revisto com base em critérios de prioridade mais objetivos, levando em conta valor de negócio, risco técnico e expectativa do cliente, além de passar a distinguir o que já foi consolidado no plano anterior do que permanece como foco de evolução para o presente plano.

Dessa forma, o plano atual não se limita a continuar o planejamento anterior, mas assume também o papel de documento de revisão e amadurecimento do projeto, evidenciando de forma direta as mudanças de requisitos propostas pelo professor e a incorporação concreta de cada uma delas ao trabalho.

13. EVOLUÇÃO ARQUITETURAL: DA ESTRUTURA DE PÁGINA ÚNICA PARA A ORGANIZAÇÃO DE MÚTIPLAS PÁGINAS

Nos estágios iniciais do projeto, a lógica da aplicação encontrava-se mais concentrada em uma estrutura de página única, com controle centralizado e forte dependência do componente principal da aplicação. Essa abordagem era adequada para a fase inicial do MVP, mas ainda mantinha navegação, estado e apresentação excessivamente acoplados, dificultando a demonstração mais madura de reuso entre páginas e a evolução arquitetural do sistema.

Para atender diretamente ao feedback recebido, foi explicitamente extraído da antiga estrutura de página única o componente de visualização de mercado, responsável por exibir cotações, variações e cards com informações resumidas das principais criptomoedas acompanhadas pela plataforma. Esse componente deixou de estar preso à renderização centralizada da tela principal e passou a ser tratado como uma unidade reutilizável, incorporada a uma organização em múltiplas páginas mais coerente com a evolução do projeto.

A decisão arquitetural de extrair esse componente foi tomada por razões técnicas objetivas. Em primeiro lugar, ele representa uma unidade funcional coesa, pois reúne responsabilidades diretamente ligadas à exibição de dados de mercado. Em segundo lugar, possui alto potencial de reutilização visual e funcional, uma vez que é relevante tanto em contextos de entrada na plataforma quanto em contextos de uso autenticado. Em terceiro lugar, depende diretamente de dados externos e, por isso, se beneficia de uma separação mais clara entre visualização, navegação e camada de serviços. Em quarto lugar, sua extração reduz acoplamento, melhora manutenção e prepara a aplicação para crescimento incremental com maior consistência estrutural.

A reutilização entre páginas ocorre de forma concreta. Na Home, o componente apresenta uma visão resumida do mercado, com foco em descoberta, contextualização e atração do usuário. Na Dashboard, o mesmo componente é reutilizado em contexto autenticado, articulado com informações adicionais de carteira, histórico e acompanhamento de ativos. Assim, a reutilização deixa de ocorrer apenas dentro de uma única tela e passa a se manifestar de forma arquiteturalmente mais madura, entre páginas distintas da aplicação.

Essa mudança evidencia que a aplicação evoluiu de um MVP com controle mais centralizado para uma estrutura de múltiplas páginas mais modular, mais manutenível e mais adequada à integração com dados externos e à continuidade do desenvolvimento.

14. IMPLEMENTAÇÃO EFETIVA DA INTEGRAÇÃO COM API

Nos planejamentos anteriores, a integração com API aparecia principalmente como intenção de sprint e meta de milestone, mas ainda sem detalhamento textual suficiente para comprovar sua implementação na versão apresentada. Para responder diretamente ao feedback recebido, este TP5 explicita de forma objetiva o funcionamento dessa integração na aplicação.

Na versão entregue, o componente de visualização de mercado passou a consumir dados provenientes de uma API externa de mercado, utilizada para obter cotações atualizadas das criptomoedas exibidas na interface. Para isso, a aplicação adota uma camada de serviços responsável por isolar a chamada HTTP da camada visual, tratar a resposta recebida, normalizar os dados e repassá-los ao componente consumidor em formato adequado para renderização.

O fluxo implementado pode ser descrito da seguinte forma. Ao acessar a página que utiliza o componente de mercado, a interface dispara a busca de dados. Em seguida, a camada de serviço consulta a API externa de cotações. Os dados retornados são convertidos para o formato utilizado internamente pela aplicação. Depois disso, o estado do componente é atualizado com os novos valores. A interface é re-renderizada automaticamente com as informações recebidas. Em caso de falha, é exibido um estado de erro com mensagem apropriada. Durante o carregamento, a interface apresenta um estado visual de loading, evitando tela estática ou sem contexto.

Essa descrição não representa apenas uma intenção de implementação, mas o registro textual do comportamento funcional da interface no protótipo entregue. Em outras palavras, os dados exibidos no bloco de mercado deixam de ser tratados apenas como conteúdo previsto em backlog e passam a compor a lógica funcional já incorporada à aplicação.

Além disso, a integração está associada diretamente ao componente reutilizado entre Home e Dashboard, o que reforça duas evidências importantes no relatório: a integração já não está isolada como planejamento futuro e a atualização da interface ocorre sobre um componente efetivamente reaproveitado em páginas distintas.

Com isso, o Marco M3 deixa de ser descrito apenas como previsão e passa a ser caracterizado, no relatório, como etapa concretizada no protótipo funcional apresentado, com consumo de dados, tratamento de carregamento, tratamento de erro e atualização reativa da interface.

15. PRIORIZAÇÃO DO BACKLOG POR VALOR DE NEGÓCIO E RISCO TÉCNICO

No planejamento anterior, o backlog já continha itens relevantes, como autenticação, carteira, histórico, camada de API, testes e melhorias de experiência do usuário. Contudo, sua priorização ainda se dava de maneira genérica, sem associação textual clara com critérios objetivos que justificassem a ordem das entregas.

Para corrigir essa limitação, a priorização passa a seguir dois critérios complementares: **valor de negócio e risco técnico**.

O valor de negócio corresponde ao impacto direto de cada funcionalidade para o usuário final e para a proposta central da plataforma. Em um sistema de compra e venda de criptomoedas, isso significa priorizar

aquilo que permite ao usuário acompanhar o mercado, autenticar-se com segurança, operar, consultar carteira e verificar seu histórico de transações.

O risco técnico corresponde à complexidade de implementação, às dependências arquiteturais e ao potencial de falha ou retrabalho associado a cada item. Funcionalidades estruturais, como camada de serviços, tratamento de erro, autenticação persistente, organização de componentes e testes, reduzem risco para todas as demais entregas e sustentam a evolução do sistema com maior estabilidade.

Com base nesses critérios, os itens foram reorganizados da seguinte forma:

Sprint 1 de evolução

BL-06 — camada de API e tratamento padronizado de erros;
BL-01 — autenticação com persistência de sessão;
BL-07 — normalização de componentes e pastas;
BL-13 — testes unitários iniciais.

Esses itens foram priorizados por reduzirem risco técnico e criarem base estrutural para as demais funcionalidades.

Sprint 2 de evolução

BL-03 — carteira detalhada;
BL-04 — histórico de transações;
BL-09 — validações de formulários;
BL-10 — loading, empty state e skeletons.

Esses itens entregam valor direto ao usuário e consolidam a experiência principal da plataforma.

Sprint 3 de evolução

BL-05 — fluxo de compra e venda;
BL-08 — custom hooks;
BL-14 — testes de componentes;
BL-12 — observabilidade.

Essa etapa aprofunda a operação central do sistema e reduz acoplamento técnico.

Sprint 4 de evolução

BL-15 — testes E2E;
BL-16 — otimizações de performance;
BL-19 — página de ativo;
BL-18 e BL-20 — documentação e padronização final.

Essa organização fortalece a aderência do backlog ao Scrum, pois transforma uma lista ampla de atividades em um plano evolutivo coerente, incremental e orientado por valor.

16. FLUXO REATIVO DE DADOS E COMUNICAÇÃO ENTRE COMPONENTES

Outro ponto aprofundado neste planejamento é a formalização do comportamento reativo da interface. Nos textos anteriores, a interatividade era descrita mais como intenção funcional do que como fluxo técnico. Aqui, esse comportamento passa a ser explicitado de forma estruturada.

O fluxo reativo da aplicação pode ser descrito da seguinte maneira. Primeiro, há a origem do dado, que pode vir da entrada do usuário, do estado local ou de um serviço externo. Em seguida, ocorre a atualização do

estado, quando uma ação do usuário ou o retorno da API modifica o estado por meio dos hooks do React. Na sequência, há a propagação reativa: a alteração do estado provoca nova renderização dos componentes dependentes. Depois, acontece a composição da interface, em que componentes filhos recebem dados e funções por props e reagem à mudança. Em seguida, a interface apresenta feedback visual coerente com o estado atual, como loading, erro, sucesso, conteúdo vazio ou conteúdo renderizado. Por fim, quando aplicável, preferências e sessão podem ser mantidas entre navegações, reforçando persistência e sincronização.

Aplicando esse fluxo ao projeto, tem-se o seguinte exemplo: o usuário acessa a Dashboard; a página dispara a busca pelas cotações; o serviço retorna os dados; o estado `marketData` é atualizado; o componente de mercado re-renderiza os cards; ao selecionar uma criptomoeda ou preencher um valor de compra, o formulário recalcula a operação; após a confirmação, os estados da carteira e do histórico são atualizados; e os componentes relacionados refletem imediatamente a alteração, sem recarregamento manual da página.

Esse encadeamento comprova o comportamento reativo exigido, pois evidencia que a interface responde a mudanças de estado e à interação entre componentes, mantendo sincronismo entre dados, lógica e apresentação. O sistema, portanto, não funciona como uma estrutura estática ou apenas navegável, mas como uma aplicação efetivamente orientada por estado e reação.

17. SÍNTESE DAS MODIFICAÇÕES REALIZADAS EM RESPOSTA AOS FEEDBACKS

Em síntese, as modificações incorporadas ao presente planejamento em resposta aos feedbacks recebidos foram:

- criação de uma seção formal de revisão do planejamento anterior;
- explicitação textual dos feedbacks e de sua incorporação;
- enumeração objetiva das mudanças de requisitos propostas pelo professor;
- identificação do componente extraído da antiga estrutura de página única;
- justificativa técnica da nova organização de múltiplas páginas;
- demonstração da reutilização do componente entre Home e Dashboard;
- formalização do consumo de API e dos estados de integração;
- revisão do backlog com priorização por valor de negócio e risco técnico;
- associação dos itens priorizados às próximas sprints;
- detalhamento formal do fluxo reativo de dados entre API, estado e componentes;
- separação mais clara entre o que foi consolidado no TP4 e o que está sendo desenvolvido neste TP5.

Dessa forma, o presente planejamento não apenas dá continuidade ao anterior, mas também corrige suas lacunas documentais e fortalece a coerência entre planejamento, arquitetura, implementação e gestão ágil. O resultado é um relatório mais rastreável, tecnicamente mais consistente e mais aderente aos critérios de avaliação do projeto.

18. REVISÃO DO SISTEMA E IDENTIFICAÇÃO DOS COMPONENTES QUE PRECISAM SER MODIFICADOS.

A revisão do sistema desenvolvido até aqui mostra que as próximas modificações não devem ser compreendidas apenas como ajustes estruturais ou reorganização de código. Cada componente listado abaixo possui impacto direto no comportamento funcional da plataforma, alterando a forma como o usuário acessa informações, executa operações e percebe a resposta do sistema.

a. Componente de autenticação (LoginForm / fluxo de login)

Esse componente precisa ser modificado para consolidar autenticação com persistência de sessão,

validação mais robusta e tratamento adequado de erro.

Impacto no comportamento do sistema:

A modificação permitirá que o usuário permaneça autenticado entre navegações e retornos à aplicação, reduzindo a necessidade de novo login a cada acesso. Também melhora o comportamento funcional do sistema ao impedir acessos indevidos a áreas restritas, exibir mensagens claras em caso de falha e tornar o fluxo de entrada mais previsível e seguro.

b. Componente de proteção de rotas (ProtectedRoute ou equivalente)

Esse componente precisa ser ajustado para controlar corretamente o acesso a páginas autenticadas, como Dashboard, carteira e histórico.

Impacto no comportamento do sistema:

Com essa modificação, páginas privadas deixarão de ficar acessíveis sem autenticação válida. Funcionalmente, isso garante coerência entre estado de login e navegação, evitando que usuários não autenticados visualizem dados internos ou acessem fluxos de compra e venda sem permissão.

c. Componente de visualização de mercado (MarketOverview, MarketCards ou equivalente)

Esse componente precisa ser refinado para consolidar o consumo de dados reais, reutilização entre páginas e atualização automática das informações exibidas.

d. Impacto no comportamento do sistema:

A consequência funcional mais importante é que as cotações, variações e indicadores de mercado passarão a ser apresentados de forma consistente tanto na Home quanto na Dashboard. Além disso, a atualização dos dados deixará a interface mais confiável para consulta, apoiando decisões do usuário com base em informações reais e não apenas simuladas.

e. Camada de serviços de mercado (marketService, apiService ou equivalente)

A camada responsável pelas requisições à API precisa ser modificada para padronizar consumo de dados, tratamento de erro, normalização da resposta e integração com os componentes visuais.

Impacto no comportamento do sistema:

Essa alteração melhora diretamente a confiabilidade funcional da plataforma. O sistema passa a reagir melhor a falhas externas, evitando travamentos ou exibição de informações inconsistentes. Também torna o comportamento da interface mais previsível em cenários de carregamento, indisponibilidade de serviço ou retorno incompleto da API.

f. Componente de Dashboard

A Dashboard precisa ser modificada para integrar de forma mais clara os blocos de mercado, carteira, histórico e futuras operações de compra e venda.

Impacto no comportamento do sistema:

A mudança tornará a Dashboard um ambiente funcional mais completo, e não apenas uma tela de exibição fragmentada. O usuário passará a ter uma leitura mais integrada do sistema, visualizando em um mesmo espaço os dados necessários para acompanhar ativos, analisar mercado e tomar decisões operacionais.

g. Componente de carteira (Portfolio, WalletSummary ou equivalente)

Esse componente precisa ser ampliado para apresentar saldo total, distribuição por ativo, desempenho e atualização após operações.

Impacto no comportamento do sistema:

Com essa modificação, a carteira deixará de ser apenas um bloco informativo básico e passará a refletir efetivamente a posição do usuário após compras e vendas. Isso altera o comportamento funcional do sistema porque a plataforma passa a responder às operações realizadas, mostrando consequências reais sobre saldo, composição e desempenho dos ativos.

h. Componente de histórico de transações (TransactionHistory)

Esse componente precisa ser modificado para registrar operações, permitir filtragem e organizar a visualização cronológica das movimentações.

i. Impacto no comportamento do sistema

A consequência funcional é que o usuário poderá acompanhar suas ações passadas dentro da plataforma, conferindo maior rastreabilidade às operações realizadas. O sistema deixa de oferecer apenas estado atual e passa também a registrar memória de uso, o que aumenta utilidade prática e confiança na navegação.

j. Componente de compra e venda (TradeForm, BuySellForm ou equivalente)

Esse é um dos componentes que mais precisam de evolução, pois deve consolidar o fluxo central da plataforma: simulação e registro de operações.

Impacto no comportamento do sistema:

Quando esse componente for concluído, o sistema deixará de ser predominantemente consultivo e passará a permitir ação direta do usuário sobre sua carteira. Essa é uma mudança funcional central, porque transforma a aplicação de um painel de acompanhamento em uma plataforma com jornada operacional mais completa.

l. Componentes de feedback visual (Loading, ErrorState, EmptyState, Skeleton)

Esses componentes precisam ser padronizados e distribuídos de forma consistente pelas telas principais.

Impacto no comportamento do sistema:

A modificação melhora a compreensão do usuário sobre o estado atual da aplicação. Em vez de parecer travado, vazio ou inconsistente, o sistema passa a comunicar claramente quando está carregando dados, quando houve falha e quando não há conteúdo disponível. Isso aumenta previsibilidade, reduz ambiguidade e melhora a experiência de uso.

m. Componente de navegação (Navbar, Sidebar, Layout)

A navegação precisa ser ajustada para refletir melhor a separação entre áreas públicas e privadas e para reforçar o fluxo entre Home, Dashboard, carteira e histórico.

Impacto no comportamento do sistema:

Essa modificação muda a jornada do usuário dentro da aplicação. A navegação passa a orientar melhor o uso do sistema, reduzindo confusão e tornando mais claro o caminho entre consulta de mercado, autenticação e áreas operacionais. Funcionalmente, isso melhora usabilidade e continuidade do fluxo.

o. Hooks e gerenciamento de estado (useAuth, useMarketData, usePortfolio ou equivalentes)

Esses elementos precisam ser revisados para encapsular lógicas reutilizáveis e sincronizar melhor dados entre componentes.

Impacto no comportamento do sistema:

O principal efeito funcional é a maior consistência da interface. Alterações feitas em um ponto do sistema passam a refletir corretamente em outros componentes dependentes, reduzindo divergências entre telas, evitando duplicação de lógica e melhorando a resposta reativa da aplicação.

Síntese do impacto das modificações

A revisão do sistema mostra que as modificações necessárias não têm apenas finalidade estrutural. Cada componente citado influencia diretamente o comportamento funcional da plataforma. Em conjunto, essas alterações produzem os seguintes efeitos principais:

- tornam o acesso mais seguro e coerente com o estado de autenticação;
- melhoram a confiabilidade da consulta de dados de mercado;
- fazem a interface reagir de forma mais clara a carregamento, erro e ausência de dados;

- permitem que carteira e histórico reflitam efetivamente as ações do usuário;
- aproximam o sistema da jornada principal esperada pelo cliente, especialmente no fluxo de compra e venda;
- transformam a aplicação em uma solução mais integrada, previsível e funcional.

Dessa forma, a enumeração dos componentes deixa de ser apenas uma lista de partes técnicas e passa a demonstrar como cada modificação interfere concretamente no uso e no comportamento do sistema.

19. REVISÃO DO BACKLOG COM FOCO NO VALOR PARA O CLIENTE

Considerando que o projeto se encontra em fase final, o backlog foi revisado com base nas novas informações levantadas ao longo do TP4 e reorganizado segundo critérios mais objetivos de priorização. Para evitar uma ordenação apenas descritiva, cada item passou a ser associado a métricas de valor mensuráveis, especialmente impacto no usuário, valor de negócio percebido pelo cliente e esforço estimado em Story Points.

A expectativa do cliente para a plataforma permanece centrada na entrega de uma jornada principal funcional e confiável: acompanhar o mercado de criptomoedas, autenticar-se com segurança, consultar carteira, visualizar histórico de transações e realizar operações de compra e venda de forma clara e previsível. Por esse motivo, os itens de maior prioridade foram definidos a partir do quanto contribuem diretamente para essa experiência principal e do custo estimado para sua implementação.

Para fins de priorização, foram adotadas as seguintes referências:

- Impacto no usuário: escala de 1 a 5, em que 1 representa baixo impacto e 5 representa impacto muito alto na experiência principal;
- Valor de negócio: escala de 1 a 5, em que 1 representa baixo valor percebido pelo cliente e 5 representa valor muito alto;
- Esforço estimado: medido em Story Points, considerando complexidade, dependências e risco técnico;
- Critério de valor mensurável: resultado objetivo esperado após a entrega do item.

19.1 O que foi desenvolvido e consolidado no TP5

No TP5, foram consolidados os seguintes pontos:

- criação de uma seção formal de revisão dos feedbacks recebidos;
- enumeração das mudanças de requisitos propostas pelo professor;
- explicitação da evolução da arquitetura da aplicação;
- extração do componente de visualização de mercado da lógica centralizada;
- demonstração de reutilização desse componente entre Home e Dashboard;
- formalização textual da integração efetiva com API externa;
- descrição do tratamento de carregamento, falha e atualização automática da interface;
- explicitação do fluxo reativo entre serviço, estado e componentes;
- reorganização do backlog com base em valor de negócio e risco técnico.

Esses itens mostram que o TP5 não se limitou a acrescentar funcionalidades isoladas. Ele consolidou a base estrutural que sustenta a próxima entrega, tornando o sistema mais coerente, reutilizável, integrado e aderente aos critérios de avaliação.

19.2 Backlog revisado e ordenado por prioridade

Prioridade	Item	Impacto no usuário	Valor de negócio	Esforço (SP)	Critério de valor mensurável	Status
1	BL-05 — Fluxo de compra e venda	5	5	8	Usuário consegue concluir uma operação válida e ver atualização imediata na carteira e no histórico	TP5
2	BL-03 — Carteira com visão detalhada por ativo	5	5	5	100% dos ativos do usuário exibem saldo, distribuição e desempenho de forma organizada	TP5
3	BL-04 — Histórico de transações com filtros	4	5	5	100% das operações confirmadas passam a aparecer no histórico com possibilidade de consulta e filtro	TP5
4	BL-01 — Autenticação com persistência de sessão	4	5	5	Sessão mantida entre navegações e recarregamento da aplicação sem novo login indevido	TP5
5	BL-02 — Estrutura básica de segurança	4	5	3	100% das rotas privadas redirecionam usuários não autenticados	TP5
6	BL-06 — Camada de API e tratamento padronizado de erros	4	4	5	Dados de mercado carregados com loading e tratamento de erro sem quebra da interface	TP4
7	BL-10 — Loading, empty state e skeletons	4	4	3	Estados assíncronos principais apresentam feedback visual claro ao usuário	TP4 parcial / TP5 finalização
8	BL-09 — Validação de formulários com mensagens amigáveis	4	4	3	Formulários inválidos são bloqueados e exibem mensagens claras em todos os campos críticos	TP5
9	BL-07 — Normalização de componentes e estrutura de pastas	3	4	3	Componentes compartilhados reutilizados entre páginas e estrutura mais consistente para manutenção	TP4
10	BL-14 — Testes de componentes	3	4	5	Componentes críticos da interface passam a ter validação automatizada dos principais comportamentos	TP5
11	BL-13 — Testes unitários	3	4	3	Funções, hooks e regras de negócio críticas passam a ter cobertura automatizada	TP5
12	BL-15 — Testes E2E dos fluxos críticos	3	4	5	Fluxo principal do usuário validado ponta a ponta sem falhas críticas	TP5
13	BL-12 — Observabilidade e captura de erros	2	3	3	Erros relevantes passam a ser registrados para facilitar depuração e estabilidade	TP5
14	BL-16 — Otimizações de performance	3	3	5	Melhoria perceptível de carregamento e fluidez nas telas principais	TP5
15	BL-18 — README técnico e documentação final	2	3	2	Sistema documentado com instruções de execução, arquitetura e decisões principais	TP5
16	BL-20 — Padronização de estilo e regras do projeto	2	3	2	Código e interface passam a seguir padrão uniforme de organização e apresentação	TP5
17	BL-19 — Página de ativo	2	3	5	Usuário consegue acessar detalhes de um ativo específico em página dedicada	TP5
18	BL-11 — Aplicativo mobile nativo	2	2	8	Criação de base mobile executável com telas nativas principais	Fora do escopo do TP5

Prioridade	Item	Impacto no usuário	Valor de negócio	Esforço (SP)	Critério de valor mensurável	Status
19	BL-17 — Gestures na versão mobile	2	2	5	Implementação de interações como pull-to-refresh e swipe actions em ambiente nativo	Fora do escopo do TP5

19.3 Síntese da revisão do backlog

A revisão do backlog mostra que o TP4 foi responsável por consolidar a base estrutural e documental do projeto, enquanto o TP5 deve concentrar-se na entrega da jornada funcional completa que o cliente espera. Com essa separação, o relatório passa a demonstrar com clareza o que já foi efetivamente amadurecido nesta etapa e o que ainda precisa ser implementado na próxima, evitando a impressão de que tudo permanece apenas em planejamento.

20. TESTES REALIZADOS COM O REACT TESTING LIBRARY

Como o projeto se aproxima da etapa final, torna-se fundamental ampliar a estratégia de testes automatizados, utilizando a React Testing Library para validar o comportamento real da interface a partir da perspectiva do usuário. Nesta etapa, o objetivo não é apenas verificar se os componentes renderizam corretamente, mas também garantir que o sistema responda de forma adequada a falhas, carregamentos assíncronos e interações mais complexas durante o uso.

A adoção da React Testing Library é adequada ao projeto porque permite testar a interface de maneira mais próxima do comportamento observado em produção. Em vez de validar detalhes internos de implementação, os testes passam a verificar elementos renderizados, mensagens exibidas, mudanças de estado e respostas da interface após ações do usuário, como cliques, preenchimento de formulários, navegação e envio de dados.

Para atender ao feedback recebido, a cobertura de testes foi ampliada em três frentes principais.

1. Testes de cenários de erro

Além dos fluxos de sucesso, o sistema deve ser testado em situações de falha. Isso inclui, por exemplo, erro de autenticação no login, indisponibilidade da API de mercado, falha no carregamento da carteira, erro ao buscar histórico de transações e validações incorretas em formulários.

Nesses casos, os testes devem verificar se:

- a mensagem de erro correta é exibida ao usuário;
- a interface não fica travada ou inconsistente;
- componentes dependentes não apresentam dados inválidos;
- o usuário recebe feedback claro sobre o problema ocorrido.

Essa ampliação é importante porque garante que o sistema não seja validado apenas em condições ideais, mas também em situações reais de uso, nas quais falhas de rede, entradas inválidas e respostas inesperadas podem acontecer.

2. Testes de estados assíncronos

Como a aplicação consome dados externos e depende de atualização de estado após requisições, os testes precisam contemplar comportamentos assíncronos. Isso envolve verificar o estado de carregamento inicial, a substituição do loading pelo conteúdo final, a renderização após retorno de API e a exibição de erro quando a resposta falha.

Com isso, os testes passam a validar, por exemplo:

- se o componente exibe loading enquanto os dados ainda não chegaram;
- se os cards de mercado aparecem apenas após a conclusão da requisição;
- se mensagens de erro substituem corretamente o conteúdo em caso de falha;
- se a interface reage de forma previsível a mudanças de estado disparadas de maneira assíncrona.

Essa abordagem é especialmente relevante no projeto porque a visualização de mercado, a dashboard e os dados de carteira dependem diretamente de integração com API e atualização reativa da interface.

3. Testes de interações complexas do usuário

A cobertura também precisa avançar para além de cliques simples e renderização inicial. A proposta é validar fluxos completos de interação, como preenchimento de login, tentativa de acesso com dados inválidos, seleção de ativo, digitação de valores de operação, envio de formulário, atualização de carteira e alteração de histórico após uma ação confirmada.

Nesses cenários, a React Testing Library deve ser utilizada para simular interações mais próximas do uso real, verificando:

- preenchimento de múltiplos campos de formulário;
- cliques sequenciais em botões e controles da interface;
- mudanças condicionais na renderização;
- atualização de componentes dependentes após uma mesma ação;
- resposta visual do sistema após operações do usuário.

Esse tipo de teste é importante porque o comportamento da plataforma não depende apenas de componentes isolados, mas da comunicação entre estado, formulários, serviços e componentes visuais.

Cenários prioritários de teste

Com base na arquitetura e nas funcionalidades do projeto, os cenários prioritários de teste passam a incluir:

Componente ou fluxo	Cenário a testar	Resultado esperado
Login	envio com credenciais válidas	autenticação concluída e redirecionamento correto

Componente ou fluxo	Cenário a testar	Resultado esperado
Login	envio com credenciais inválidas	mensagem de erro exibida sem quebrar a interface
MarketOverview	carregamento inicial dos dados	exibição de loading antes do conteúdo
MarketOverview	retorno bem-sucedido da API	renderização dos cards com dados recebidos
MarketOverview	falha na API	exibição de mensagem de erro ou estado alternativo
Dashboard	carregamento assíncrono de informações	atualização da interface após retorno dos dados
TradeForm	preenchimento incorreto	bloqueio da operação e exibição de validação
TradeForm	operação confirmada	atualização coerente da carteira e do histórico
TransactionHistory	ausência de dados	exibição de empty state
Componentes visuais	mudança entre loading, erro e conteúdo	transição correta entre estados

Impacto da ampliação da cobertura de testes

A ampliação da cobertura de testes traz impacto direto na qualidade final do projeto. Em vez de validar apenas renderizações básicas, a aplicação passa a ser testada em situações mais próximas do uso real. Isso aumenta a confiabilidade da interface, reduz regressões, melhora a robustez diante de falhas e fortalece a evidência de que o sistema responde adequadamente tanto em cenários esperados quanto em situações excepcionais.

Dessa forma, o uso da React Testing Library deixa de ser apenas uma exigência instrumental e passa a compor uma estratégia mais madura de validação da aplicação, contemplando comportamento funcional, estados assíncronos, tratamento de erro e interações complexas do usuário.

O projeto agora já está com React Testing Library + Vitest funcionando e com testes iniciais.

O que foi implementado

Adicionadas dependências de teste no devDependencies e scripts:

- test: vitest run
- test:watch: vitest

- Arquivo: [package.json](c:\Users\marcu\Documents\VSCode\Plataforma Criptomoeda\package.json)

Configurado o Vitest no Vite (jsdom + setupFiles):

- Arquivo: [vite.config.js](c:\Users\marcu\Documents\VSCode\Plataforma Criptomoeda\vite.config.js)

Criados setup global dos testes:

- [src/test/setup.js](c:\Users\marcu\Documents\VSCode\Plataforma Criptomoeda\src\test\setup.js)

Testes iniciais:

- [src/App.test.jsx](c:\Users\marcu\Documents\VSCode\Plataforma Criptomoeda\src\App.test.jsx)
- [src/components/MarketOverview.test.jsx](c:\Users\marcu\Documents\VSCode\Plataforma Criptomoeda\src\components\MarketOverview.test.jsx)

README atualizado com comandos de teste:

- [README.md](c:\Users\marcu\Documents\VSCode\PlataformaCriptomoeda\README.md)

21. MECANISMO DE NAVEGAÇÃO ADOTADO, ROTAS DEFINIDAS E ESTRUTURA DE LAYOUT

A navegação da aplicação foi estruturada com base em uma abordagem de roteamento no front-end, adequada ao modelo de aplicação web em React. Para organizar a transição entre páginas e separar áreas públicas e autenticadas do sistema, foi adotado o React Router, responsável por definir as rotas da aplicação, controlar a navegação sem recarregamento completo da página e manter a coerência entre URL, estado de autenticação e conteúdo exibido.

Tecnicamente, o mecanismo de navegação é iniciado a partir do componente raiz da aplicação, no qual o sistema é encapsulado por um provedor de rotas, normalmente por meio do BrowserRouter. A partir desse ponto, as páginas passam a ser registradas em uma estrutura de Routes e Route, permitindo que cada caminho da URL corresponda a um componente específico da interface. Essa abordagem substitui a navegação puramente centralizada em um único componente e torna a arquitetura mais adequada a uma aplicação de múltiplas páginas. No contexto do projeto, a organização de rotas foi definida da seguinte forma:

Rota	Página / Componente	Tipo de acesso	Finalidade
/	Home	Pública	Apresentar a proposta da plataforma e visão inicial do mercado
/login	Login	Pública	Permitir autenticação do usuário

Rota	Página / Componente	Tipo de acesso	Finalidade
/dashboard	Dashboard	Privada	Exibir visão principal do usuário autenticado
/portfolio	Carteira	Privada	Apresentar saldo, distribuição de ativos e desempenho
/history	Histórico	Privada	Exibir histórico de transações
/asset/:id	Página de ativo	Privada ou futura	Detalhar informações de uma criptomoeda específica

A distinção entre rotas públicas e privadas é importante para o comportamento funcional do sistema. As rotas públicas permanecem acessíveis sem autenticação, enquanto as rotas privadas dependem da validação da sessão do usuário. Para isso, a navegação pode ser associada a um componente intermediário de proteção, como `ProtectedRoute`, responsável por verificar se o usuário está autenticado antes de liberar o acesso ao conteúdo solicitado. Caso a autenticação não esteja válida, o sistema redireciona o usuário para a tela de login, preservando a segurança da navegação e a coerência entre sessão e acesso.

Além da definição das rotas, a aplicação também foi organizada com uma **estrutura de layout** para padronizar a apresentação visual entre páginas. Em vez de repetir manualmente os mesmos elementos em cada tela, a navegação foi articulada com componentes de layout reutilizáveis, como barra superior, menu lateral, rodapé e contêiner principal de conteúdo. Essa estrutura pode ser representada por um layout público, utilizado em páginas como Home e Login, e por um layout autenticado, utilizado em páginas como Dashboard, Carteira e Histórico.

Do ponto de vista técnico, essa composição é normalmente realizada com o uso de `Outlet`, recurso do React Router que permite que um layout pai renderize, dentro de sua estrutura, a página filha correspondente à rota acessada. Com isso, elementos persistentes de navegação, como `Navbar`, `Sidebar` e áreas de conteúdo, permanecem organizados de forma consistente, enquanto apenas o conteúdo central é substituído conforme a rota muda.

Essa arquitetura de navegação traz ganhos relevantes para o projeto. Em primeiro lugar, melhora a separação entre responsabilidades, pois cada página passa a ter função mais bem definida. Em segundo lugar, favorece reutilização de layout e componentes compartilhados. Em terceiro lugar, reforça a experiência do usuário, pois a navegação ocorre de forma fluida, sem recarregamento completo da aplicação. Por fim, facilita a evolução da plataforma, permitindo a inclusão de novas páginas e fluxos com menor acoplamento estrutural.

De forma sintética, o mecanismo adotado combina três elementos principais: React Router para controle de rotas, rotas públicas e privadas para organizar acesso e layouts reutilizáveis para padronizar a estrutura visual da navegação. Assim, o sistema deixa de operar como uma interface centralizada e passa a apresentar uma navegação tecnicamente organizada, coerente com a proposta de múltiplas páginas da aplicação.

22. ATUALIZAÇÕES FINAIS DECORRENTES DAS ÚLTIMAS ALTERAÇÕES DO PROJETO

Após a revisão final das mudanças levantadas com o professor e das alterações efetivamente realizadas no sistema, este documento passa a registrar de forma ainda mais explícita o impacto dessas mudanças nas histórias de usuário, nos componentes do sistema e na delimitação do escopo de entrega. As alterações mais recentes concentraram-se em três pontos centrais: refinamento das histórias com rastreabilidade direta aos feedbacks, consolidação dos componentes que ainda exigem modificação para completar a jornada principal do cliente e explicitação formal de que aplicativo mobile nativo e gestures permanecem como evolução futura, não como funcionalidade implementada nesta entrega.

Com isso, o arquivo final de entrega não apenas preserva todo o conteúdo anteriormente consolidado, mas também passa a evidenciar, de forma complementar, quais requisitos foram amadurecidos nas últimas revisões e como esses ajustes afetam o planejamento técnico e funcional da plataforma.

22.1 Histórias de usuário revisadas a partir das últimas alterações

As histórias de usuário abaixo consolidam as últimas alterações discutidas no projeto e tornam mais visível a ligação entre feedback recebido, modificação incorporada e situação da funcionalidade no escopo atual:

ID	História de usuário revisada	Ajuste incorporado	Situação na entrega
HU-01	Como investidor da plataforma, eu quero visualizar cotações e variações reais das criptomoedas, para acompanhar o mercado e apoiar minhas decisões.	Passa a reforçar explicitamente a integração efetiva com API externa e a reutilização do componente de mercado entre Home e Dashboard.	Consolidada
HU-02	Como usuário autenticado, eu quero acessar uma Dashboard organizada e protegida, para acompanhar meus dados com segurança e continuidade de navegação.	Foi alinhada ao uso de rotas privadas, layout autenticado e necessidade de persistência de sessão.	Consolidação prioritária do TP5
HU-03	Como usuário da plataforma, eu quero consultar minha carteira com saldo total, distribuição por ativo e desempenho, para entender minha posição atual.	Foi refinada para refletir atualização após operações e vínculo direto com o backlog da carteira.	Consolidação prioritária do TP5
HU-04	Como usuário da plataforma, eu quero consultar meu histórico de transações com filtros e organização cronológica, para auditar minhas movimentações.	Passa a explicitar rastreabilidade das operações e integração com a jornada operacional da plataforma.	Consolidação prioritária do TP5
HU-05	Como usuário da plataforma, eu quero simular e confirmar operações de compra e venda, para agir diretamente sobre minha carteira.	Foi reforçada como núcleo funcional da expectativa do cliente, exigindo atualização imediata da carteira e do histórico.	Maior prioridade funcional do TP5
HU-06	Como cliente da plataforma, eu quero uma versão mobile nativa do sistema, para utilizar a solução em ambiente próprio de dispositivos móveis.	Foi mantida como requisito identificado pelo professor, porém explicitamente registrada como não implementada nesta entrega.	Evolução futura / fora do escopo atual
HU-07	Como usuário mobile, eu quero interagir com a aplicação por meio de gestures, como pull-to-	Foi registrada como dependente da implementação prévia do aplicativo mobile	Evolução futura / fora do escopo atual

	refresh e swipe actions, para ter uma experiência nativa mais fluida.	nativo, não podendo ser tratada como funcionalidade já entregue.	
--	---	--	--

23. RASTREABILIDADE FINAL ENTRE COMPONENTES, MODIFICAÇÕES E BACKLOG

Para complementar a revisão do sistema, a tabela a seguir consolida os principais componentes que exigem modificação, o tipo de ajuste necessário, o item do backlog correspondente e a prioridade associada. Essa visão integrada fortalece a rastreabilidade entre arquitetura, implementação e planejamento:

Componente	Modificação principal	Backlog relacionado	Prioridade
LoginForm / fluxo de login	Persistência de sessão, validação robusta, tratamento de erro e previsibilidade do acesso.	BL-01 / BL-09	Alta
ProtectedRoute	Restringir acesso a Dashboard, carteira e histórico conforme autenticação válida.	BL-02	Alta
MarketOverview / MarketCards	Consolidar consumo de dados reais, reuso entre páginas e atualização automática.	BL-06 / BL-07	Alta
marketService / apiService	Padronizar requisições, loading, erro e normalização da resposta da API.	BL-06	Alta
Dashboard	Integrar mercado, carteira, histórico e operações em uma visão única mais coerente.	BL-03 / BL-04 / BL-05	Alta
Portfolio / WalletSummary	Exibir saldo, distribuição por ativo, desempenho e reflexo imediato das operações.	BL-03	Alta
TransactionHistory	Registrar operações, permitir filtros e organizar cronologia das movimentações.	BL-04	Alta
TradeForm / BuySellForm	Permitir simulação, validação, confirmação e atualização imediata da jornada operacional.	BL-05 / BL-09	Alta
Loading / ErrorState / EmptyState / Skeleton	Padronizar feedback visual para cenários assíncronos e ausência de dados.	BL-10	Média
Navbar / Sidebar / Layout	Reforçar separação entre áreas públicas e privadas e orientar a navegação do usuário.	BL-07	Média
Hooks de estado e serviços	Encapsular lógicas reutilizáveis e sincronizar dados entre componentes dependentes.	BL-08 / BL-13 / BL-14	Média

24. DELIMITAÇÃO FINAL DO ESCOPO DE ENTREGA E EXPECTATIVA DO CLIENTE

A expectativa do cliente para a plataforma permanece centrada em uma jornada principal funcional, confiável e coerente: acompanhar o mercado de criptomoedas, autenticar-se com segurança, consultar carteira, visualizar histórico e operar compra e venda com feedback claro da interface. As últimas alterações registradas neste documento reforçam justamente essa direção, pois deixam mais nítido o que já foi amadurecido na solução e o que ainda constitui evolução posterior.

24.1 Elementos que passam a constar de forma consolidada nesta entrega

Passam a constar de forma consolidada nesta entrega: a revisão rastreável dos feedbacks do professor; a enumeração objetiva das mudanças de requisitos; a formalização da extração e reutilização do componente de mercado; a explicitação da integração com API e do fluxo reativo; a ampliação da estratégia de testes com React Testing Library; a definição do mecanismo de navegação com React Router; e a priorização do backlog com foco na jornada principal esperada pelo cliente.

24.2 Elementos que permanecem como evolução futura

Permanecem como evolução futura, e não como funcionalidade já implementada nesta entrega, a versão mobile nativa da aplicação e o uso de gestures específicos de ambiente móvel. Esses itens foram preservados no backlog para manter aderência ao feedback do professor e transparência quanto ao escopo real do trabalho, evitando que requisitos ainda não implementados sejam apresentados como concluídos.

Dessa forma, o documento final de entrega mantém integralmente o conteúdo já produzido e, adicionalmente, incorpora as últimas modificações relevantes para tornar o relatório mais claro, mais rastreável e mais alinhado tanto aos critérios de avaliação quanto à expectativa funcional do cliente.